CMSC132: Wildcards and Generics

Chapter 7

By Naren Ayinala

May 2026

Contents

1 Why Generics Exist **2**

2 Generic Classes and Multiple Type Parameters **2**

2.1 Diamond Operator vs Raw Type 2

3 Generic Methods **3**

4 Generics, Inheritance, and Invariance **3**

5 Unbounded Wildcards: `<?>` **3**

6 Upper Bounds: `<? extends Type>` **4**

7 Lower Bounds: `<? super Type>` **4**

8 PECS: Producer Extends, Consumer Super **4**

9 Bounded Type Parameters and Comparability **5**

10 Type Erasure and Generic Arrays **5**

11 Autoboxing and Unboxing **5**

12 Practice with Answers **5**

1 Why Generics Exist

Generics allow us to write classes, interfaces, and methods that work with different reference types while keeping compile-time type safety. Instead of creating a separate box class for strings and integers, we create one `Box<T>` and choose the type when using it. This avoids many casts and catches mistakes earlier.

```
import java.util.*;
List<String> names = new ArrayList<>();
names.add("Ada");
// names.add(42); // Compile-time error: expected String
String first = names.get(0); // no cast needed
```

Remember: Java generic type arguments must be reference types. Use `Integer`, not `int`; `Double`, not `double`. Generic types are invariant by default, a rule we will explain below.

2 Generic Classes and Multiple Type Parameters

The type parameter `T` is a placeholder. You can name it something else, but `T`, `E`, `K`, and `V` are common conventions. The compiler checks uses of the placeholder against the chosen type argument.

```
class Box<T> {
    private T value;
    public Box(T value) { this.value = value; }
    public T get() { return value; }
    public void set(T value) { this.value = value; }
}

Box<String> greeting = new Box<>("hello");
String s = greeting.get();
Box<Integer> count = new Box<>(10);
```

We can also use more than one type parameter. A key–value pair is a natural example.

```
class Pair<K, V> {
    private final K key;
    private final V value;
    Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }
    K getKey() { return key; }
    V getValue() { return value; }
}
Pair<String, Integer> p = new Pair<>("score", 95);
```

2.1 Diamond Operator vs Raw Type

`new Box<>(10)` uses the **diamond operator**: Java infers the type from context. `new Box(10)` uses a **raw type**: it drops generic checking and should usually be avoided. Empty angle brackets and omitted angle brackets are *not* the same thing.

3 Generic Methods

A generic method declares its own type parameter *before the return type*. It can appear inside a class that is not generic.

```
class Util {
    public static <T> T first(List<T> items) {
        return items.get(0);
    }
    public static <T> void printBoth(T a, T b) {
        System.out.println(a + " " + b);
    }
}
String name = Util.first(List.of("Ada", "Grace"));
```

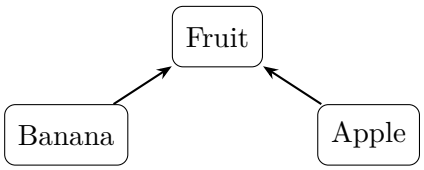
The method's type parameter is inferred from the arguments. A subtle point: `<T> void f(T a,T b)` does *not* necessarily require both arguments to have the exact same runtime class. The compiler can infer a common supertype such as `Object`. A named `T` means the two positions share one inferred type variable, not that their runtime classes must be identical.

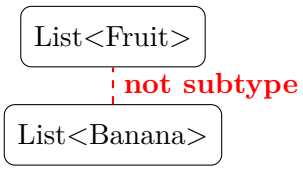
4 Generics, Inheritance, and Invariance

Suppose `Banana extends Fruit`. Then a `Banana` is a `Fruit`, but a `List<Banana>` is *not* a `List<Fruit>`. Why? If Java allowed it, someone could add an `Apple` into the underlying banana list.

```
class Fruit {}
class Banana extends Fruit {}
class Apple extends Fruit {}

Fruit f = new Banana(); // valid polymorphism
List<Banana> bananas = new ArrayList<>();
// List<Fruit> fruits = bananas; // invalid: invariant
```





Arrays differ: they are covariant, so `Fruit[] a = new Banana[2]` compiles. But assigning an `Apple` into that array throws `ArrayStoreException` at runtime.

5 Unbounded Wildcards: <?>

A wildcard says the exact element type is unknown. `List<?>` can refer to a list of strings, integers, or other objects. We can read elements as `Object`, but cannot add a non-null element because we do not know the list's element type.

```
static void printAll(List<?> list) {
    for (Object x : list) System.out.println(x);
```

```
    // list.add("hello"); // not safe
    list.add(null); // legal, though rarely useful
}
printAll(List.of(1, 2, 3));
printAll(List.of("a", "b"));
```

Two separate wildcards are independent unknown types. If you need to express that several positions share the same type, a named type parameter is often more appropriate.

6 Upper Bounds: `<? extends Type>`

An upper-bounded wildcard means the element type is some unknown subtype of the bound. For example, `List<? extends Number>` could refer to `List<Integer>` or `List<Double>`. It is safe to read values as `Number`, but unsafe to insert a particular `Number` subtype.

```
static double sum(List<? extends Number> nums) {
    double total = 0;
    for (Number n : nums) total += n.doubleValue();
    return total;
}
System.out.println(sum(List.of(1, 2, 3))); // 6.0
System.out.println(sum(List.of(1.5, 2.5))); // 4.0
```

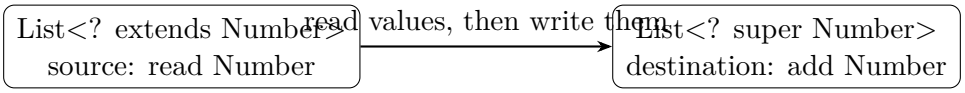
7 Lower Bounds: `<? super Type>`

A lower-bounded wildcard means the element type is some unknown *supertype* of the bound. For example, `List<? super Integer>` could be a list of `Integer`, `Number`, or `Object`. It is safe to add `Integer` values, but a retrieved value can only be treated as `Object` without further checks.

```
static void addCounts(List<? super Integer> dest) {
    dest.add(10);
    dest.add(20);
    Object first = dest.get(0); // Object is safe
}
List<Number> numbers = new ArrayList<>();
addCounts(numbers);
```

8 PECS: Producer Extends, Consumer Super

This is a useful memory trick. If a structure **produces** values for us to read, `extends` is often right. If it **consumes** values we insert, `super` is often right.




```
static <T> void copy(List<? extends T> src,
                     List<? super T> dest) {
    for (T value : src) dest.add(value);
```

```
}
List<Integer> ints = List.of(1, 2);
List<Number> nums = new ArrayList<>();
copy(ints, nums);
```

9 Bounded Type Parameters and Comparability

A named type parameter can have a bound. The following method requires values that can compare themselves to other values of the same type.

```
static <T extends Comparable<? super T>> T max(T a, T b) {
   return a.compareTo(b) >= 0 ? a : b;
}
System.out.println(max(3, 8)); // 8
System.out.println(max("cat", "dog")); // dog
```

10 Type Erasure and Generic Arrays

Most generic type information is erased at runtime. Consequently, Java does not allow `new T[10]` or `new List<String>[10]`. A common alternative is to use a `List<T>` rather than an array. Some implementations create an `Object[]` internally and cast elements on retrieval; unchecked array casts need careful justification and are not universally safe.

```
class Bag<T> {
   private final List<T> data = new ArrayList<>();
   void add(T item) { data.add(item); }
   T get(int i) { return data.get(i); }
}
```

11 Autoboxing and Unboxing

Autoboxing converts a primitive to its wrapper; unboxing converts the wrapper back to a primitive. These conversions are convenient, but unboxing a null wrapper throws `NullPointerException`.

```
List<Integer> nums = new ArrayList<>();
nums.add(5); // boxes int -> Integer
int x = nums.get(0); // unboxes Integer -> int
Integer missing = null;
// int y = missing; // NullPointerException
```

12 Practice with Answers

Q1. Why is `List<Dog>` not a subtype of `List<Animal>`?
Answer: Allowing that assignment would make it possible to insert another animal subtype into a list intended only for dogs.

Q2. What is the safe return type from `List<? super Integer>.get(0)`?
Answer: `Object`.

Q3. Can you call `add(3.14)` on `List<? extends Number>`?
Answer: No. Its actual element type might be `Integer`.

Q4. Why is `new Box<>(5)` different from `new Box(5)`?
Answer: The former infers a type argument; the latter is a raw-type use that weakens checking.